

Week2_예습과제

CH02 사이킷런으로 시작하는 머신러닝

1. 사이킷런 소개와 특징

사이킷런의 특징 : 머신러닝을 위한 다양한 알고리즘과 개발을 위한 프레임워크와 API 제공

2. 첫번째 머신러닝 만들어 보기-붓꽃 품종 예측하기

분류 :

대표적인 지도학습 방법, 학습 데이터 세트 → 다양한 피처와 분류 결정값 ⇒ 레이블 데이터
→ 학습 → 미지의 데이터 세트(테스트 데이터 세트)에서 예측

1. 피처들과 데이터 값이 어떻게 구성돼 있는지 확인하기 위해 **DataFrame**으로 변환

2. 학습용데이터//테스트용데이터 분리

`X_train, X_test, y_train, y_test = train_test_split(피처데이터세트, 레이블데이터세트, 테스트데이터세트 비율, 난수발생값)`

`X_train` : 학습피처데이터세트를 반환

`X_test` : 테스트용피처데이터세트

`y_train` : 학습용레이블데이터세트

`y_test` : 테스트용레이블데이터세트

3. 분류알고리즘으로 모델 학습과 예측 수행, 예측 평가

객체생성 > 학습수행 > 학습 완료된 객체에서 테스트데이터세트로 예측수행 > 예측 평가

3. 사이킷런의 기반 프레임워크 익히기

`Classifier` 클래스 : 분류 알고리즘 구현 클래스,

`Regressor` 클래스 : 회귀 알고리즘 구현 클래스

*분류, 회귀는 대표적인 지도학습

`Estimator` 클래스 : `Classifier`, `Regressor` 통칭한 클래스, `fit()`, `predict()` 구현

`fit()` : ML모델학습

predict() : 학습된 모델 예측

이외에도 비지도학습에 이용되는 클래스와 메소드가 사이킷런에 존재

머신러닝 모델을 구축하는 주요 프로세스는

피처 처리, ML알고리즘 학습/예측 수행, 모델 평가 단계를 반복적으로 수행

datasets 안 데이터 세트들 : 분류, 회귀 연습을 위한 예제용도의 데이터세트 // 분류, 클러스터링을 위한 표본데이터로 생성될 수 있는 데이터 세트

표본 데이터 생성기는 보통 딕셔너리 형태로 되어 있으며 키는 보통 data, target, target_name, feature_names, DESCR로 구성되어 있음.

data	피처의 데이터 세트	넘파이배열(ndarray)
target	분류 → 레이블값 // 회귀 → 숫자 결괏값 데이터세트	넘파이배열
target_names	개별 레이블 이름	넘파이배열 or 리스트타입
feature_names	피처 이름	넘파이배열 or 리스트타입
DESCR	데이터세트에 대한 설명과 각 피처의 설명	스트링타입

4. Model Selection 모듈 소개

학습/테스트 데이터 세트 분리 :

train_test_split(피처데이터세트, 레이블데이터세트) :

이후 파라미터) test_size : 전체 데이터에서 테스트 데이터 세트 크기를 얼마로 샘플링할건지

train_size : 전체 데이터에서 학습용 데이터 세트 크기를 얼마로 샘플링건지

shuffle : 데이터를 분리하기 전, 전체 데이터를 미리 섞을지

random_state : 호출할 때마다 동일한 학습/테스트용 데이터 세트를 생성하기 위해 주어지는 난수값, random_state를 지정하지 않으면 수행 때마다 다른 학습/테스트데이터 세트를 생성

*이 함수의 리턴값은 튜플 형태.

교차 검증 :

과적합, 데이터 편중 방지를 위해 교차 검증을 사용. 별도의 여러 세트로 구성된 학습 데이터 세트/검증 데이터 세트에서 학습과 평가 수행.

학습 데이터 학습 > 교차검증 기반의 1차평가(학습데이터, 검증데이터) > 최종 테스트 데이터 세트에 적용

K 폴드 교차 검증 :

K개의 데이터 폴드 세트를 만들어 K번만큼 각 폴드 세트에 학습, 검증평가 반복 수행, 학습데이터세트와 검증데이터세트를 점진적으로 변경하면서 K번째까지 학습과 검증을 수행하는 것.

⇒ 폴드세트 설정 > for루프에서 반복으로 학습&테스트데이터 인덱스 추출 > 반복 학습 & 예측 수행해 예측 성능 반환

Stratified K 폴드 :

불균형한 분포도를 가진 레이블 데이터 집합을 위한 K폴드 방식.

*불균형한 분포도를 가진 레이블 데이터 집합 : 특정레이블 값이 특이하게 많거나 적어서 값의 분포가 치우치는 것.

일반적으로 분류에서는 Stratified K폴드 이용 ↔ 회귀에서는 Stratified K폴드 지원x(⇒ 회귀의 결정값은 이산값 형태의 레이블이 아닌 연속된 숫자값이기 때문)

`cross_val_score(estimator, X, y=None, scoring=None, cv=None)` :

KFold의 과정 한번에 수행.

`estimator` : Classifier 또는 Regressor클래스 의미

`X` : 피쳐 데이터 세트

`y` : 레이블 데이터 세트

`scoring` : 예측 성능 평가 지표

`cv` : 교차 검증 폴드 수

메소드 수행 후 `scoring` 파라미터로 지정된 성능지표측정값을 배열 형태로 리턴.

이때 `cv`에 지정된 횟수만큼 평가 결괏값이 배열로 생김.

classifier(분류) → `cross_val_score()` → stratified K폴드 방식

regressor (회귀) → `cross_val_score()` → K폴드 방식

교차검증과 최적 하이퍼파라미터 튜닝을 한번에 :

*하이퍼파라미터 : 머신러닝 알고리즘을 구성하는 주요 구성요소, 이 값을 조정해 알고리즘의 예측 성능 개선 가능

GridSearchCV API :

Classifier나 Regressor와 같은 알고리즘에 사용되는 하이퍼파라미터를 순차적으로 입력하면서 편리하게 최적의 파라미터를 도출할 수 있는 방안 제공

데이터 세트를 교차 검증을 위한 학습/테스트 세트로 자동 분할한 뒤 하이퍼 파라미터 그리드에 기술된 모든 파라미터를 순차적으로 적용, 최적의 파라미터 탐색.

5. 데이터 전처리

null 값은 고정된 다른 값으로 변환

문자열 피처는 보통 카테고리형 피처// 텍스트형 피처로 나뉘는데 카테고리형 피처는 코드값으로 // 텍스트형 피처는 피처 벡터화 or 삭제

데이터 인코딩 :

레이블 인코딩 // 원-핫 인코딩

레이블 인코딩 : 카테고리 피처(문자열값) → 코드형 숫자값(숫자형카테고리값), LabelEncoder클래스로 구현, 객체를 생성 후 fit(), transform()을 호출해 레이블 인코딩 수행 /선형회귀 알고리즘에선 적용해선 안됨(가중치부여됨).

원-핫 인코딩 : 행 형태로 되어 있는 피처의 고유 값을 열 형태로 차원 변환, 고유 값에 해당하는 칼럼에만 1표시, 나머지 칼럼엔 0표시

OneHotEncoder클래스로 변환 가능, 이때 1) 2차원 데이터가 필요하며 2)다시 toarray()메서드를 이용해 밀집행렬로 변환해야.

판다스에서는 get_dummies()를 통해 바로 원-핫 인코딩 가능

피처 스케일링과 정규화 :

피처 스케일링 : 서로 다른 변수의 값 범위를 일정 수준으로 맞추는 작업, 표준화/정규화가 있음.

표준화 : $(x - \text{평균}) / \text{표준편차}$

데이터의 피쳐 평균이 0, 분산이 1인 값(가우시안 분포)으로 변환됨.

정규화 : $(x - \text{최솟값}) / (\text{최댓값} - \text{최솟값})$

서로 다른 피쳐의 크기를 0에서 1사이로 통일하기 위해 크기를 변환함.

벡터 정규화 : 개별 벡터의 크기를 맞추기 위해 변환.

StandardScaler :

표준화를 쉽게 지원하기 위한 클래스. StandardScaler 객체 생성 후 fit(), transform() 메서드에 변환 대상 피쳐 데이터 세트를 입력하고 호출.

MinMaxScaler :

데이터 값을 0과 1 사이의 범위 값으로 변환.

학습데이터, 테스트 데이터의 스케일링 변환 시 주의점 :

위 Scaler 클래스들로 Scaler 객체를 이용해 데이터 스케일링 변환 시 fit(), transform(), fit_transform() 메서드 이용.

fit() : 데이터 변환을 위한 기준 정보 설정(데이터 세트의 최댓값/최솟값 설정 등)

transform() : 이렇게 설정된 정보를 이용해 데이터 변환

fit_transform() : fit(), transform() 한 번에

⇒ 머신러닝 모델은 학습데이터를 기반으로 학습되기 때문에 반드시 테스트 데이터는 학습데이터의 스케일링 기준에 따라야!

⇒ 전체 데이터 스케일링 변환 적용 후 학습/테스트 데이터 분리 or 테스트 데이터 변환 시 fit()이나 fit_transform() 적용하지 않고 학습데이터로 이미 fit()된 Scaler 객체 이용해 transform()으로 변환 (ex. scaler.transform(테스트데이터))

CH03 평가

머신러닝 프로세스 : 데이터 가공/변환 > 모델 학습/예측 > 평가(Evaluation)

분류의 평가방법 :

정확도(Accuracy), 오차행렬(Confusion Matrix), 정밀도(Precision), 재현율(Recall), F1스코어, ROC AUC

*분류는 이진/멀티분류로 나뉨.

1. 정확도

정확도(Accuracy) = 예측결과가 동일한 데이터 건수/전체 예측 데이터 건수 :

불균형한 레이블 데이터 세트에서는 성능 수치로 사용해서는 안됨.

⇒ 여러 분류 평가 지표와 함께 적용돼야 함.

2. 오차 행렬

오차행렬(confusion matrix, 혼동행렬) : 학습된 분류 모델이 예측을 수행하면서 얼마나 헛갈리고 있는지 함께 보여줌. 이진 분류의 예측 오류가 얼마인지 & 어떠한 유형의 예측 오류가 발생하고 있는지를 함께 나타내는 지표

		예측 클래스 (Predicted Class)	
		Negative(0)	Positive(1)
실제 클래스 (Actual Class)	Negative(0)	TN (True Negative)	FP (False Positive)
	Positive(1)	FN (False Negative)	TP (True Positive)

실제 레이블 클래스 값과 예측 레이블 클래스 값이 어떠한 유형을 가지고 매핑되는지를 나타냄. 예측 클래스와 실제 클래스의 값 유형에 따라 결정되는 TN, FP, FN, TP 형태로 오차 행렬의 4분면을 채울 수 있음.

True : 예측클래스 값 = 실제 클래스 값

Negative : 예측값이 Negative

⇒ TN : 예측값을 Negative 값 0이라고 예측했는데 실제 값도 Negative 값 0

confusion_matrix() API 제공, Classifier의 성능을 측정할 수 있는 주요 지표인 정확도 (Accuracy), 정밀도(Precision), 재현율(Recall) 값을 알 수 있음.

정확도 = $(TN + TP) / (TN + FP + FN + TP)$

⇒ 불균형한 데이터 세트에선 정확도보다 정밀도, 재현율이 선호됨.

3. 정밀도와 재현율

정밀도 = $TP / (FP + TP)$:

예측을 Positive로 한 대상 중에 예측과 실제 값이 Positive로 일치하는 데이터의 비율

재현율 = $TP / (FN + TP)$:

실제 값이 Positive인 대상 중에 예측과 실제 값이 Positive로 일치하는 데이터의 비율, 민감도 또는 TPR이라고도 불림

재현율이 더 중요한 지표인 경우 : 실제 p 양성 데이터를 n로 잘못 판단하게 되면 업무상 큰 영향이 발생하는 경우

정밀도가 더 중요한 지표인 경우 : 실제 n 음성 데이터 예측을 p로 잘못 판단하게 되면 업무상 큰 영향이 발생하는 경우

정밀도/재현율 트레이드 오프 : 정밀도나 재현율 중 한 쪽을 강제로 높이면 다른 하나의 수치는 떨어지기 쉬움.

predict_proba() : 학습이 완료된 사이킷런 Classifier 객체에서 호출이 가능하며 테스트 피쳐 데이터 세트 입력 시 테스트 피쳐 레코드의 개별 클래스 예측 확률을 ndarray로 반환. 이때 ndarray는 첫번째 칼럼이 클래스 값 0에 대한 예측 확률, 두번째 칼럼이 클래스 값 1에 대한 예측 확률

⇒ positive 예측의 임계값을 변경함에 따라 정밀도와 재현율의 수치가 변경됨. 임계값의 이러한 변경은 두 수치가 상호 보완될 수 있는 수준에서 적용돼야.

4. F1 스코어

F1스코어 : 정밀도와 재현율을 결합한 지표

$F1 = 2 * \text{precision} * \text{recall} / (\text{precision} + \text{recall})$

f1_score() API 제공

5. ROC 곡선과 AUC

ROC 곡선 : FPR(False Positive Rate)가 변할 때 TPR(True Positive Rate)가 어떻게 변하는지 나타내는 곡선. FPR $\Rightarrow$ x축 & TPR $\Rightarrow$ y축

*TPR = 재현율 = 민감도

TNR = 특이성 $\Rightarrow$ FPR = 1-특이도

분류 결정 임계값(Threshold)을 0에서 1까지 변화시키면서 FPR이 바뀔 때 TPR이 어떻게 변하는지 그래프로 표현.

ROC 곡선이 그래프 중앙의 대각선으로부터 좌측 상단으로 멀리 꺾여 올라갈수록 우수한 모델

AUC : ROC 곡선 아래쪽의 면적을 구한 수치. 1에 가까울수록 완벽한 성능 의미. 무작위 예측인 경우 0.5의 값을 가짐.